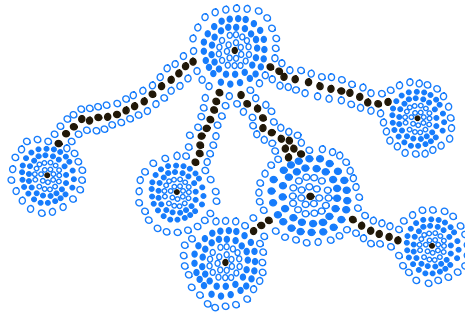




Contents

11.1 Binary Search Trees	460
11.1.1 Searching Within a Binary Search Tree	461
11.1.2 Insertions and Deletions	463
11.1.3 Java Implementation	466
11.1.4 Performance of a Binary Search Tree	470
11.2 Balanced Search Trees	472
11.2.1 Java Framework for Balancing Search Trees	475
11.3 AVL Trees	479
11.3.1 Update Operations	481
11.3.2 Java Implementation	486
11.4 Splay Trees	488
11.4.1 Splaying	488
11.4.2 When to Splay	492
11.4.3 Java Implementation	494
11.4.4 Amortized Analysis of Splaying ★	495
11.5 (2,4) Trees	500
11.5.1 Multiway Search Trees	500
11.5.2 (2,4)-Tree Operations	503
11.6 Red-Black Trees	510
11.6.1 Red-Black Tree Operations	512
11.6.2 Java Implementation	522
11.7 Exercises	525

11.1 Binary Search Trees

In Chapter 8 we introduced the tree data structure and demonstrated a variety of applications. One important use is as a *search tree* (as described on page 338). In this chapter, we use a search-tree structure to efficiently implement a *sorted map*. The three most fundamental methods of a map (see Section 10.1.1) are:

- `get( $k$ )`: Returns the value v associated with key k , if such an entry exists; otherwise returns null.
- `put( $k, v$ )`: Associates value v with key k , replacing and returning any existing value if the map already contains an entry with key equal to k .
- `remove( $k$ )`: Removes the entry with key equal to k , if one exists, and returns its value; otherwise returns null.

The sorted map ADT includes additional functionality (see Section 10.3), guaranteeing that an iteration reports keys in sorted order, and supporting additional searches such as `higherEntry( $k$ )` and `subMap( $k_1, k_2$ )`.

Binary trees are an excellent data structure for storing entries of a map, assuming we have an order relation defined on the keys. In this chapter, we define a *binary search tree* as a *proper binary tree* (see Section 8.2) such that each internal position p stores a key-value pair (k, v) such that:

- Keys stored in the left subtree of p are less than k .
- Keys stored in the right subtree of p are greater than k .

An example of such a binary search tree is given in Figure 11.1. Notice that the leaves of the tree serve only as “placeholders.” Their use as sentinels simplifies the presentation of several of our search and update algorithms. With care, they can be represented as null references in practice, thereby reducing the number of nodes in half (since there are more leaves than internal nodes in a proper binary tree).

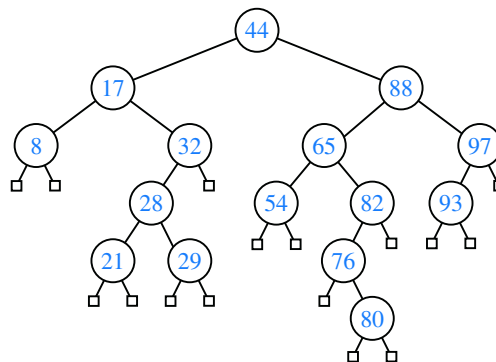


Figure 11.1: A binary search tree with integer keys. We omit the display of associated values in this chapter, since they are not relevant to the order of entries within a search tree.

11.1.1 Searching Within a Binary Search Tree

The most important consequence of the structural property of a binary search tree is its namesake search algorithm. We can attempt to locate a particular key in a binary search tree by viewing it as a decision tree (recall Figure 8.5). In this case, the question asked at each internal position p is whether the desired key k is less than, equal to, or greater than the key stored at position p , which we denote as $\text{key}(p)$. If the answer is “less than,” then the search continues in the left subtree. If the answer is “equal,” then the search terminates successfully. If the answer is “greater than,” then the search continues in the right subtree. Finally, if we reach a leaf, then the search terminates unsuccessfully. (See Figure 11.2.)

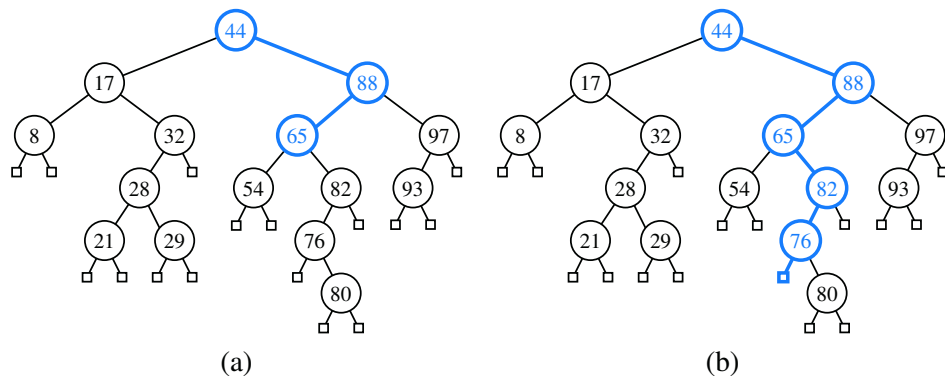


Figure 11.2: (a) A successful search for key 65 in a binary search tree; (b) an unsuccessful search for key 68 that terminates at the leaf to the left of the key 76.

We describe this approach in Code Fragment 11.1. If key k occurs in a subtree rooted at p , a call to $\text{TreeSearch}(p, k)$ results in the position at which the key is found. For an unsuccessful search, the TreeSearch algorithm returns the final leaf explored on the search path (which we will later make use of when determining where to insert a new entry in a search tree).

Algorithm $\text{TreeSearch}(p, k)$:

```

if  $p$  is external then
    return  $p$                                 {unsuccessful search}
else if  $k == \text{key}(p)$  then
    return  $p$                                 {successful search}
else if  $k < \text{key}(p)$  then
    return  $\text{TreeSearch}(\text{left}(p), k)$           {recur on left subtree}
else {we know that  $k > \text{key}(p)$ }
    return  $\text{TreeSearch}(\text{right}(p), k)$         {recur on right subtree}

```

Code Fragment 11.1: Recursive search in a binary search tree.

Analysis of Binary Tree Searching

The analysis of the worst-case running time of searching in a binary search tree T is simple. Algorithm `TreeSearch` is recursive and executes a constant number of primitive operations for each recursive call. Each recursive call of `TreeSearch` is made on a child of the previous position. That is, `TreeSearch` is called on the positions of a path of T that starts at the root and goes down one level at a time. Thus, the number of such positions is bounded by $h + 1$, where h is the height of T . In other words, since we spend $O(1)$ time per position encountered in the search, the overall search runs in $O(h)$ time, where h is the height of the binary search tree T . (See Figure 11.3.)

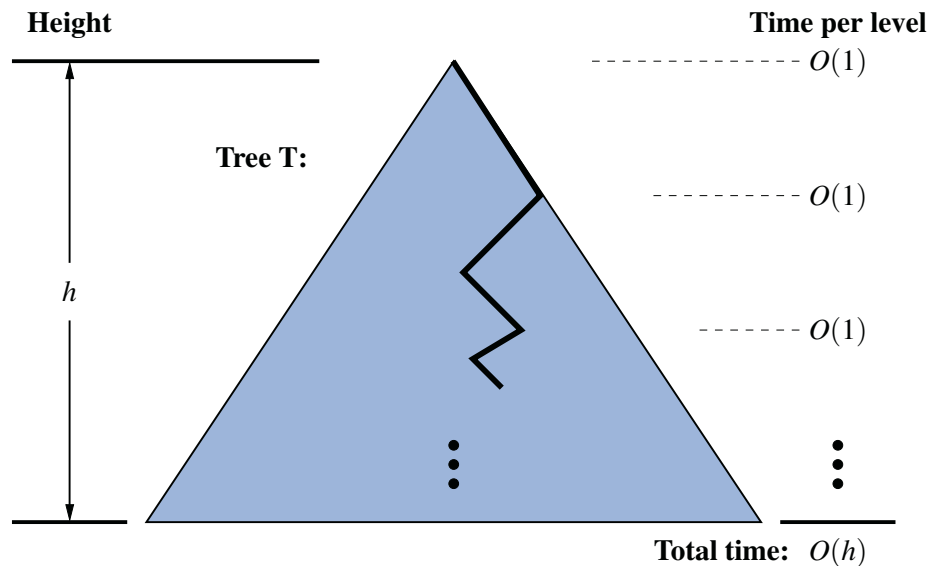


Figure 11.3: Illustrating the running time of searching in a binary search tree. The figure uses a standard visualization shortcut of a binary search tree as a big triangle and a path from the root as a zig-zag line.

In the context of the sorted map ADT, the search will be used as a subroutine for implementing the `get` method, as well as for the `put` and `remove` methods, since each of these begins by trying to locate an existing entry with the given key. We will later demonstrate how to implement sorted map operations, such as `lowerEntry` and `higherEntry`, by navigating within the tree after performing a standard search. All of these operations will run in worst-case $O(h)$ time for a tree with height h .

Admittedly, the height h of T can be as large as the number of entries, n , but we expect that it is usually much smaller. Later in this chapter we will show various strategies to maintain an upper bound of $O(\log n)$ on the height of a search tree T .

11.1.2 Insertions and Deletions

Binary search trees allow implementations of the put and remove operations using algorithms that are fairly straightforward, although not trivial.

Insertion

The map operation $\text{put}(k, v)$ begins with a search for an entry with key k . If found, that entry's existing value is reassigned. Otherwise, the new entry can be inserted into the underlying tree by expanding the leaf that was reached at the end of the failed search into an internal node. The binary search-tree property is sustained by that placement (note that it is placed exactly where a search would expect it). Let us assume a proper binary tree supports the following update operation:

expandExternal(p, e): Stores entry e at the external position p , and expands p to be internal, having two new leaves as children.

We can then describe the `TreelInsert` algorithm with the pseudocode given in in Code Fragment 11.2. An example of insertion into a binary search tree is shown in Figure 11.4.

Algorithm `TreelInsert( $k, v$ ):`

Input: A search key k to be associated with value v

$p = \text{TreeSearch}(\text{root}(), k)$

if $k == \text{key}(p)$ **then**

 Change p 's value to (v)

else

$\text{expandExternal}(p, (k, v))$

Code Fragment 11.2: Algorithm for inserting a key-value pair into a map that is represented as a binary search tree.

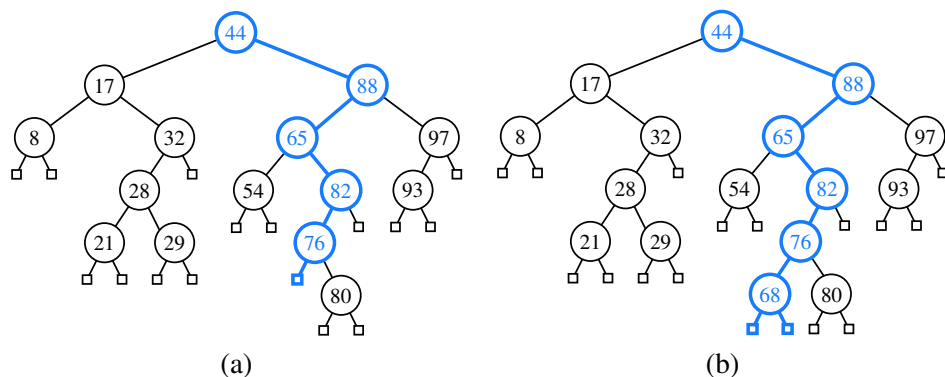


Figure 11.4: Insertion of an entry with key 68 into the search tree of Figure 11.2. Finding the position to insert is shown in (a), and the resulting tree is shown in (b).

Deletion

Deleting an entry from a binary search tree is a bit more complex than inserting a new entry because the position of an entry to be deleted might be anywhere in the tree (as opposed to insertions, which always occur at a leaf). To delete an entry with key k , we begin by calling `TreeSearch(root(), k)` to find the position p storing an entry with key equal to k (if any). If the search returns an external node, then there is no entry to remove. Otherwise, we distinguish between two cases (of increasing difficulty):

- If at most one of the children of position p is internal, the deletion of the entry at position p is easily implemented (see Figure 11.5). Let position r be a child of p that is internal (or an arbitrary child, if both are leaves). We will remove p and the leaf that is r 's sibling, while promoting r upward to take the place of p . We note that all remaining ancestor-descendant relationships that remain in the tree after the operation existed before the operation; therefore, the binary search-tree property is maintained.
- If position p has two children, we cannot simply remove the node from the tree since this would create a “hole” and two orphaned children. Instead, we proceed as follows (see Figure 11.6):
 - We locate position r containing the entry having the greatest key that is strictly less than that of position p (its so-called *predecessor* in the ordering of keys). That predecessor will always be located in the rightmost internal position of the left subtree of position p .
 - We use r 's entry as a replacement for the one being deleted at position p . Because r has the immediately preceding key in the map, any entries in p 's right subtree will have keys greater than r and any other entries in p 's left subtree will have keys less than r . Therefore, the binary search-tree property is satisfied after the replacement.
 - Having used r 's entry as a replacement for p , we instead delete the node at position r from the tree. Fortunately, since r was located as the rightmost internal position in a subtree, r does not have an internal right child. Therefore, its deletion can be performed using the first (and simpler) approach.

As with searching and insertion, this algorithm for a deletion involves the traversal of a single path downward from the root, possibly moving an entry between two positions of this path, and removing a node from that path and promoting its child. Therefore, it executes in time $O(h)$ where h is the height of the tree.

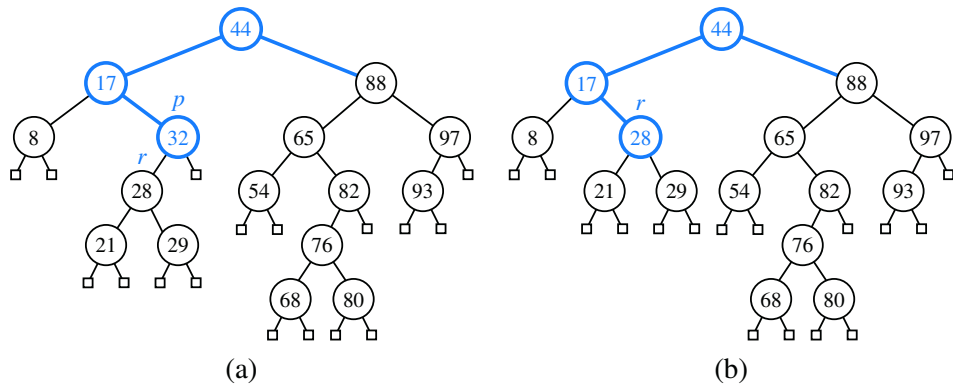


Figure 11.5: Deletion from the binary search tree of Figure 11.4b, where the entry to delete (with key 32) is stored at a position p with one child r : (a) before the deletion; (b) after the deletion.

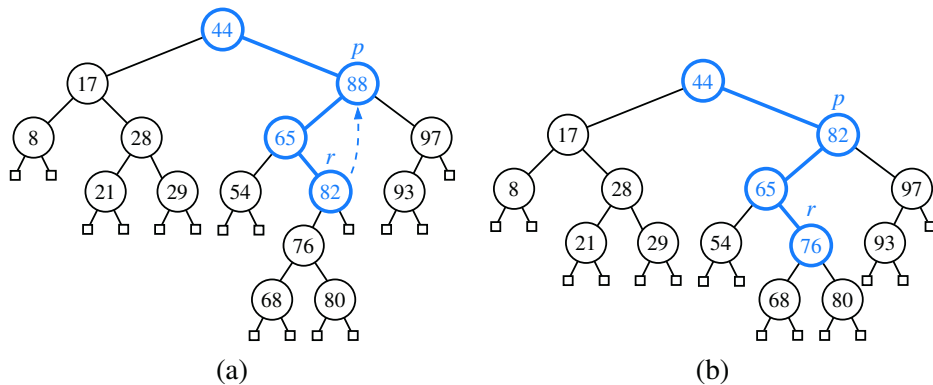


Figure 11.6: Deletion from the binary search tree of Figure 11.5b, where the entry to delete (with key 88) is stored at a position p with two children, and replaced by its predecessor r : (a) before the deletion; (b) after the deletion.

11.1.3 Java Implementation

In Code Fragments 11.3 through 11.6 we define a `TreeMap` class that implements the sorted map ADT while using a binary search tree for storage. The `TreeMap` class is declared as a child of the `AbstractSortedMap` base class, thereby inheriting support for performing comparisons based upon a given (or default) `Comparator`, a nested `MapEntry` class for storing key-value pairs, and concrete implementations of methods `keySet` and `values` based upon the `entrySet` method, which we will provide. (See Figure 10.2 on page 406 for an overview of our entire map hierarchy.)

For representing the tree structure, our `TreeMap` class maintains an instance of a subclass of the `LinkedBinaryTree` class from Section 8.3.1. In this implementation, we choose to represent the search tree as a *proper* binary tree, with explicit leaf nodes in the binary tree as sentinels, and map entries stored only at internal nodes. (We leave the task of a more space-efficient implementation to Exercise P-11.55.)

The `TreeSearch` algorithm of Code Fragment 11.1 is implemented as a private recursive method, `treeSearch( $p, k$ )`. That method either returns a position with an entry equal to key k , or else the last position that is visited on the search path. The method is not only used for all of the primary map operations, `get( $k$ )`, `put( $k, v$ )`, and `remove( $k$ )`, but for most of the sorted map methods, as the final internal position visited during an unsuccessful search has either the greatest key less than k or the least key greater than k .

Finally, we note that our `TreeMap` class is designed so that it can be subclassed to implement various forms of *balanced* search trees. We discuss the balancing framework more thoroughly in Section 11.2, but there are two aspects of the design that impact the code presented in this section. First, our tree member is technically declared as an instance of a `BalanceableBinaryTree` class, which is a specialization of the `LinkedBinaryTree` class; however, we rely only on the inherited behaviors in this section. Second, our code is peppered with calls to presumed methods named `rebalanceAccess`, `rebalanceInsert`, and `rebalanceDelete`; these methods do not do anything in this class, but they serve as *hooks* that can later be customized.

We conclude with a brief guide to the organization of our code.

Code Fragment 11.3: Beginning of `TreeMap` class, including constructors, `size` method, and `expandExternal` and `treeSearch` utilities.

Code Fragment 11.4: Map operations `get( $k$ )`, `put( $k, v$ )`, and `remove( $k$ )`.

Code Fragment 11.5: Sorted map ADT methods `lastEntry()`, `floorEntry( $k$ )`, and `lowerEntry( $k$ )`, and protected utility `treeMax`. Symmetric methods `firstEntry()`, `ceilingEntry( $k$ )`, `higherEntry( $k$ )`, and `treeMin` are provided online.

Code Fragment 11.6: Support for producing an iteration of all entries (method `entrySet` of the map ADT), or of a selected range of entries (method `subMap( $k_1, k_2$ )` of the sorted map ADT).


```

1  /** An implementation of a sorted map using a binary search tree. */
2  public class TreeMap<K,V> extends AbstractSortedMap<K,V> {
3      // To represent the underlying tree structure, we use a specialized subclass of the
4      // LinkedBinaryTree class that we name BalanceableBinaryTree (see Section 11.2).
5      protected BalanceableBinaryTree<K,V> tree = new BalanceableBinaryTree<>();
6
7      /** Constructs an empty map using the natural ordering of keys. */
8      public TreeMap() {
9          super(); // the AbstractSortedMap constructor
10         tree.addRoot(null); // create a sentinel leaf as root
11     }
12     /** Constructs an empty map using the given comparator to order keys. */
13     public TreeMap(Comparator<K> comp) {
14         super(comp); // the AbstractSortedMap constructor
15         tree.addRoot(null); // create a sentinel leaf as root
16     }
17     /** Returns the number of entries in the map. */
18     public int size() {
19         return (tree.size() - 1) / 2; // only internal nodes have entries
20     }
21     /** Utility used when inserting a new entry at a leaf of the tree */
22     private void expandExternal(Position<Entry<K,V>> p, Entry<K,V> entry) {
23         tree.set(p, entry); // store new entry at p
24         tree.addLeft(p, null); // add new sentinel leaves as children
25         tree.addRight(p, null);
26     }
27
28     // Omitted from this code fragment, but included in the online version of the code,
29     // are a series of protected methods that provide notational shorthands to wrap
30     // operations on the underlying linked binary tree. For example, we support the
31     // protected syntax root() as shorthand for tree.root() with the following utility:
32     protected Position<Entry<K,V>> root() { return tree.root(); }
33
34     /** Returns the position in p's subtree having given key (or else the terminal leaf).*/
35     private Position<Entry<K,V>> treeSearch(Position<Entry<K,V>> p, K key) {
36         if (isExternal(p))
37             return p; // key not found; return the final leaf
38         int comp = compare(key, p.getElement());
39         if (comp == 0)
40             return p; // key found; return its position
41         else if (comp < 0)
42             return treeSearch(left(p), key); // search left subtree
43         else
44             return treeSearch(right(p), key); // search right subtree
45     }

```

Code Fragment 11.3: Beginning of a TreeMap class based on a binary search tree.

```

46  /** Returns the value associated with the specified key (or else null). */
47  public V get(K key) throws IllegalArgumentException {
48      checkKey(key); // may throw IllegalArgumentException
49      Position<Entry<K,V>> p = treeSearch(root(), key);
50      rebalanceAccess(p); // hook for balanced tree subclasses
51      if (isExternal(p)) return null; // unsuccessful search
52      return p.getElement().getValue(); // match found
53  }
54  /** Associates the given value with the given key, returning any overridden value.*/
55  public V put(K key, V value) throws IllegalArgumentException {
56      checkKey(key); // may throw IllegalArgumentException
57      Entry<K,V> newEntry = new MapEntry<>(key, value);
58      Position<Entry<K,V>> p = treeSearch(root(), key);
59      if (isExternal(p)) { // key is new
60          expandExternal(p, newEntry);
61          rebalanceInsert(p); // hook for balanced tree subclasses
62          return null;
63      } else { // replacing existing key
64          V old = p.getElement().getValue();
65          set(p, newEntry);
66          rebalanceAccess(p); // hook for balanced tree subclasses
67          return old;
68      }
69  }
70  /** Removes the entry having key k (if any) and returns its associated value. */
71  public V remove(K key) throws IllegalArgumentException {
72      checkKey(key); // may throw IllegalArgumentException
73      Position<Entry<K,V>> p = treeSearch(root(), key);
74      if (isExternal(p)) { // key not found
75          rebalanceAccess(p); // hook for balanced tree subclasses
76          return null;
77      } else {
78          V old = p.getElement().getValue();
79          if (isInternal(left(p)) && isInternal(right(p))) { // both children are internal
80              Position<Entry<K,V>> replacement = treeMax(left(p));
81              set(p, replacement.getElement());
82              p = replacement;
83          } // now p has at most one child that is an internal node
84          Position<Entry<K,V>> leaf = (isExternal(left(p)) ? left(p) : right(p));
85          Position<Entry<K,V>> sib = sibling(leaf);
86          remove(leaf);
87          remove(p); // sib is promoted in p's place
88          rebalanceDelete(sib); // hook for balanced tree subclasses
89          return old;
90      }
91  }

```

Code Fragment 11.4: Primary map operations for the TreeMap class.

```

92  /** Returns the position with the maximum key in subtree rooted at Position p. */
93  protected Position<Entry<K,V>> treeMax(Position<Entry<K,V>> p) {
94      Position<Entry<K,V>> walk = p;
95      while (isInternal(walk))
96          walk = right(walk);
97      return parent(walk);           // we want the parent of the leaf
98  }
99  /** Returns the entry having the greatest key (or null if map is empty). */
100 public Entry<K,V> lastEntry() {
101     if (isEmpty()) return null;
102     return treeMax(root()).getElement();
103 }
104 /** Returns the entry with greatest key less than or equal to given key (if any). */
105 public Entry<K,V> floorEntry(K key) throws IllegalArgumentException {
106     checkKey(key);                // may throw IllegalArgumentException
107     Position<Entry<K,V>> p = treeSearch(root(), key);
108     if (isInternal(p)) return p.getElement(); // exact match
109     while (!isRoot(p)) {
110         if (p == right(parent(p)))
111             return parent(p).getElement(); // parent has next lesser key
112         else
113             p = parent(p);
114     }
115     return null;                  // no such floor exists
116 }
117 /** Returns the entry with greatest key strictly less than given key (if any). */
118 public Entry<K,V> lowerEntry(K key) throws IllegalArgumentException {
119     checkKey(key);                // may throw IllegalArgumentException
120     Position<Entry<K,V>> p = treeSearch(root(), key);
121     if (isInternal(p) && isInternal(left(p)))
122         return treeMax(left(p)).getElement(); // this is the predecessor to p
123     // otherwise, we had failed search, or match with no left child
124     while (!isRoot(p)) {
125         if (p == right(parent(p)))
126             return parent(p).getElement(); // parent has next lesser key
127         else
128             p = parent(p);
129     }
130     return null;                  // no such lesser key exists
131 }

```

Code Fragment 11.5: A sample of the sorted map operations for the TreeMap class. The symmetrical utility, treeMin, and public methods firstEntry, ceilingEntry, and higherEntry are available online.

```

132  /** Returns an iterable collection of all key-value entries of the map. */
133  public Iterable<Entry<K,V>> entrySet() {
134      ArrayList<Entry<K,V>> buffer = new ArrayList<>(size());
135      for (Position<Entry<K,V>> p : tree.inorder())
136          if (isInternal(p)) buffer.add(p.getElement());
137      return buffer;
138  }
139  /** Returns an iterable of entries with keys in range [fromKey, toKey). */
140  public Iterable<Entry<K,V>> subMap(K fromKey, K toKey) {
141      ArrayList<Entry<K,V>> buffer = new ArrayList<>(size());
142      if (compare(fromKey, toKey) < 0) // ensure that fromKey < toKey
143          subMapRecurse(fromKey, toKey, root(), buffer);
144      return buffer;
145  }
146  private void subMapRecurse(K fromKey, K toKey, Position<Entry<K,V>> p,
147                          ArrayList<Entry<K,V>> buffer) {
148      if (isInternal(p))
149          if (compare(p.getElement(), fromKey) < 0)
150              // p's key is less than fromKey, so any relevant entries are to the right
151              subMapRecurse(fromKey, toKey, right(p), buffer);
152          else {
153              subMapRecurse(fromKey, toKey, left(p), buffer); // first consider left subtree
154              if (compare(p.getElement(), toKey) < 0) { // p is within range
155                  buffer.add(p.getElement()); // so add it to buffer, and consider
156                  subMapRecurse(fromKey, toKey, right(p), buffer); // right subtree as well
157              }
158          }
159  }

```

Code Fragment 11.6: TreeMap operations supporting iteration of the entire map, or a portion of the map with a given key range.

11.1.4 Performance of a Binary Search Tree

An analysis of the operations of our TreeMap class is given in Table 11.1. Almost all operations have a worst-case running time that depends on h , where h is the height of the current tree. This is because most operations rely on traversing a path from the root of the tree, and the maximum path length within a tree is proportional to the height of the tree. Most notably, our implementations of map operations `get`, `put`, and `remove`, and most of the sorted map operations, each begins with a call to the `treeSearch` utility. Similar paths are traced when searching for the minimum or maximum entry in a subtree, a task used when finding a replacement during a deletion or in finding the overall first or last entry in the map. An iteration of the entire map is accomplished in $O(n)$ time using an inorder traversal of the underlying tree, and the recursive `subMap` implementation can be shown to run in $O(s + h)$ worst-case bound for a call that reports s results (see Exercise C-11.34).

Method	Running Time
size, isEmpty	$O(1)$
get, put, remove	$O(h)$
firstEntry, lastEntry	$O(h)$
ceilingEntry, floorEntry, lowerEntry, higherEntry	$O(h)$
subMap	$O(s + h)$
entrySet, keySet, values	$O(n)$

Table 11.1: Worst-case running times of the operations for a TreeMap. We denote the current height of the tree with h , and the number of entries reported by subMap as s . The space usage is $O(n)$, where n is the number of entries stored in the map.

A binary search tree T is therefore an efficient implementation of a map with n entries only if its height is small. In the best case, T has height $h = \lceil \log(n + 1) \rceil - 1$, which yields logarithmic-time performance for most of the map operations. In the worst case, however, T has height n , in which case it would look and feel like an ordered list implementation of a map. Such a worst-case configuration arises, for example, if we insert entries with keys in increasing or decreasing order. (See Figure 11.7.)

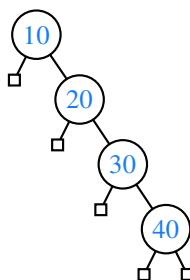


Figure 11.7: Example of a binary search tree with linear height, obtained by inserting entries in increasing order of their keys.

We can nevertheless take comfort that, on average, a binary search tree with n keys generated from a random series of insertions and removals of keys has expected height $O(\log n)$; the justification of this statement is beyond the scope of the book, requiring careful mathematical language to precisely define what we mean by a random series of insertions and removals, and sophisticated probability theory.

In applications where one cannot guarantee the random nature of updates, it is better to rely on variations of search trees, presented in the remainder of this chapter, that guarantee a *worst-case* height of $O(\log n)$, and thus $O(\log n)$ worst-case time for searches, insertions, and deletions.